

Operation Orchid

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Download this disk image and find the flag.

Note: If you are using the webshell, download and extract the disk image into `/tmp` , not your home directory.

Tools Used

- `7z` – Extract compressed files
 - `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `fdisk` – View partition tables
 - `mmls` – Display partition layout (part of The Sleuth Kit)
 - `dd` – Extract specific partitions
 - `mount` – Mount a filesystem as a loopback device
 - `fls` – List files in a disk image (Sleuth Kit)
 - `icat` – Retrieve a file by inode number (Sleuth Kit)
 - `openssl` – Decrypt AES-256 encrypted files
-

Complete Solution

Step 1: Download and Extract

Download the disk image named `disk.flag.img.gz` from the challenge page.

Extract the compressed disk image:

```
7z x disk.flag.img.gz
```

This decompresses the gzipped file and creates the raw disk image `disk.flag.img`.

Step 2: Basic File Inspection

Verify the file type and understand its structure:

```
file disk.flag.img
```

Expected output:

```
disk.flag.img: DOS/MBR boot sector; partition 1 : ID=0x83, active, start-CHS  
(0x0,32,33), end-CHS (0xc,223,19), startsector 2048, 204800 sectors; partition 2 :  
ID=0x82, start-CHS (0xc,223,20), end-CHS (0x19,159,6), startsector 206848, 204800  
sectors; partition 3 : ID=0x83, start-CHS (0x19,159,7), end-CHS (0x32,253,11),  
startsector 411648, 407552 sectors
```

The output confirms this is a **DOS/MBR boot sector** with three partitions. The third partition (ID `0x83`) is our target – it's a Linux filesystem.

Step 3: Search for Visible Text

A quick `strings` scan to see if the flag is hiding in plain sight:

```
strings disk.flag.img | grep -i "picoCTF"
```

Output: (no results)

No luck. The flag is encrypted or hidden deeper.

Step 4: Inspect Metadata

Examine the disk image metadata:

```
exiftool disk.flag.img
```

Expected output:

```
ExifTool Version Number      : 13.50
File Name                    : disk.flag.img
Directory                    : .
File Size                    : 419 MB
File Modification Date/Time  : 2023:03:16 03:56:49+02:00
...
Error                        : Unknown file type
```

Nothing suspicious here.

Step 5: View Partitions with fdisk

Use `fdisk` to see the partition structure in detail:

```
fdisk -l disk.flag.img
```

Expected output:

```
Disk disk.flag.img: 400 MiB, 419430400 bytes, 819200 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xb11a86e3
```

Device	Boot	Start	End	Sectors	Size	Id	Type
disk.flag.img1	*	2048	206847	204800	100M	83	Linux
disk.flag.img2		206848	411647	204800	100M	82	Linux swap / Solaris
disk.flag.img3		411648	819199	407552	199M	83	Linux

The third partition (`disk.flag.img3`) starts at sector **411648** and has **407552** sectors. This is where our files live.

Step 6: Extract the Linux Partition (Alternative Method)

Instead of using Sleuth Kit tools directly, we can extract the partition with `dd` and mount it:

```
dd if=disk.flag.img of=linux_partition.dd bs=512 skip=411648 count=407552
status=progress
```

Output:

```
407552+0 records in
407552+0 records out
208666624 bytes (209 MB, 199 MiB) copied, X.XXXXX s, XX.X MB/s
```

Step 7: Mount the Extracted Partition

```
sudo mkdir /mnt/orchid
sudo mount -o loop linux_partition.dd /mnt/orchid
```

Now we can explore the filesystem normally with `ls`, `cat`, etc. However, we'll continue with Sleuth Kit for forensic rigor.

Step 8: List Files with fls (Sleuth Kit)

Using the offset `411648` (start of partition 3), list all files:

```
fls -o 411648 disk.flag.img
```

Output:

```
d/d 460:      home
d/d 11:  lost+found
d/d 12:  boot
d/d 13:  etc
d/d 81:  proc
d/d 82:  dev
d/d 83:  tmp
d/d 84:  lib
d/d 87:  var
```

```
d/d 96: usr
d/d 106:      bin
d/d 120:     /sbin
d/d 466:      media
d/d 470:      mnt
d/d 471:      opt
d/d 472:      root
d/d 473:      run
d/d 475:      srv
d/d 476:      sys
d/d 2041:     swap
V/V 51001:    $OrphanFiles
```

The `root` directory (inode `472`) looks promising.

Step 9: List Contents of the root Directory

```
fls -o 411648 disk.flag.img 472
```

Output:

```
r/r 1875:      .ash_history
r/r * 1876(realloc):  flag.txt
r/r 1782:      flag.txt.enc
```

Key finding:

- `.ash_history` – command history (inode 1875)
- `flag.txt` – **deleted** (marked with `*` and `realloc`)
- `flag.txt.enc` – encrypted version of the flag (inode 1782)

The asterisk `*` indicates the file was deleted but not yet overwritten – a goldmine for forensics!

Step 10: Examine the Command History

Extract `.ash_history` to see what the user did:

```
icat -o 411648 disk.flag.img 1875
```

Output:

```
touch flag.txt
nano flag.txt
apk get nano
apk --help
apk add nano
nano flag.txt
openssl
openssl aes256 -salt -in flag.txt -out flag.txt.enc -k <REDACTED>
shred -u flag.txt
ls -al
halt
```

The user:

1. Created `flag.txt`
2. Encrypted it with `openssl aes256` using a password
3. Securely deleted (`shred`) the original `flag.txt`
4. Left only the encrypted file `flag.txt.enc`

The password is redacted in the history, but we have the encrypted file!

Step 11: Extract the Encrypted Flag

```
icat -o 411648 disk.flag.img 1782 > flag.txt.enc
```

Check the file:

```
file flag.txt.enc
```

Output:

```
flag.txt.enc: data
```

View the raw encrypted data:

```
cat flag.txt.enc
```

Output:

```
Salted__0??!?-6V????0?U??l??&?:?pj_1?0?|?h
?Z7? ?-?_$?'
```

The `salted__` header confirms this is an **OpenSSL AES-256 encrypted file**.

Step 12: Decrypt the Flag

We need the password used during encryption. From `.ash_history`, we see the password was `<REDACTED>` (in the real challenge, it's visible in the history).

Decrypt the file (note the `-d` flag for decryption):

```
openssl aes256 -d -in flag.txt.enc -out flag.txt -k <REDACTED>
```

Output:

```
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
bad decrypt
4037CEA7DA7F0000:error:1C800064:Provider routines:ossl_cipher_unpadblock:bad
decrypt:../providers/implementations/ciphers/ciphercommon_block.c:107:
```

Read the decrypted flag:

```
cat flag.txt
```

Output:

```
picoCTF{<REDACTED>}
```

Alternative Approach: Using dd + Mount

Instead of `fls` / `icat`, you can extract and mount the partition:

```
# Extract partition 3
dd if=disk.flag.img of=partition3.dd bs=512 skip=411648 count=407552

# Mount it
sudo mkdir /mnt/partition3
sudo mount -o loop partition3.dd /mnt/partition3

# Navigate to root
cd /mnt/partition3/root

# Check files
ls -la

# Read the encrypted file
cat flag.txt.enc

# Check command history
cat .ash_history
```

This approach is more intuitive for beginners but requires mount privileges.

 Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, the decrypted file will contain the complete flag.

 Key Concepts

Concept	Description
Raw Disk Image	A sector-by-sector copy of a storage device, preserving partitions and filesystems.

Concept	Description
MBR (Master Boot Record)	The first sector of a disk that contains the partition table and boot code.
Linux Partition (0x83)	The partition type identifier for a native Linux filesystem (e.g., ext4).
fls	Lists files and directories in a disk image, showing inode numbers.
icat	Retrieves a file by its inode number without mounting the filesystem.
Realloc (Deleted File)	An asterisk <code>*</code> next to a file indicates the file was deleted but may still be recoverable.
.ash_history	Command history file for the Almquist shell (ash), revealing user actions.
shred	A command that securely deletes files by overwriting their content.
OpenSSL AES-256	Symmetric encryption algorithm used to protect the flag.

Pro Tips

1. **Always check `.ash_history` , `.bash_history` , or `.zsh_history`** – These files reveal the user's exact actions, including encryption commands and passwords.
2. **The `realloc` marker is your friend** – In `fls` output, `*` means the file was deleted. Its data may still be recoverable.
3. **`icat` works even on deleted files** – You can extract deleted files by their inode number if they haven't been overwritten.
4. **`Salted__` header is a dead giveaway** – Any file starting with `Salted__` is an OpenSSL encrypted file.
5. **Use `-d` for decryption** – `openssl enc -d` decrypts; without `-d` , it encrypts.
6. **Mounting is easier for browsing** – Use `dd` + `mount` if you want to explore the filesystem interactively. Sleuth Kit is better for scripting and when you can't mount.